

Adaptive Progressive Retrieval Attention: Factorized Control of Search and Native-K/V Working Sets

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 23, 2026

Abstract

Long-context memory is not free merely because it is addressable. Progressive Retrieval Attention (PRA) must decide what part of a request is the query, how to represent it, how broadly to search, and how much native key/value (K/V) state to expose to the frozen language model. We study this as a factorized control problem over interpretation, search, and admission rather than as one scalar “effort” level.

The primary experiment evaluates 792 valid actions per example-seed unit on frozen HotpotQA and QASPER routing traces. At matched complete-chain quality, the minimum-cost factorized oracle reduces HotpotQA cost from 34.18 to 19.98 and matched QASPER cost from 16.11 to 8.95. The result is simple: difficult queries often need one expensive control, such as more roots, rather than an expensive bundle. On QASPER the sparse oracle recovers .917 of required evidence with .796 precision while admitting 10.9% of candidates.

Prediction is harder than optimization. Five-seed independent, conservative, query-hash, and autoregressive controllers cost less than the robust profile selector only because they under-allocate; none captures the oracle frontier. Retrieval representation is likewise heterogeneous: fixed, interpretable-cascade, learned-joint, and per-example-oracle root/successor policies reach .030, .048, .032, and .143 recall, respectively. Observable method selection remains unresolved.

We then test whether richer interpretation and post-root state justify more controller complexity. Semantic graph facets enlarge the legal action surface but are specialized rather than universal. A matched 29,700-action study finds no stage-wise oracle advantage on the measured surface, while learned root-state callbacks underperform the conservative one-shot policy. An eight-example native-K/V decode study confirms that selected memory reaches and changes a frozen consumer, but is not a benchmark QA result. The supported runtime is therefore conservative: explicit query roles when available, independent control surfaces, robust initial allocation, one bounded correction, and research hooks for richer adaptation.

1 Introduction

Long-context memory is not free merely because it is addressable. A model with an indexed memory still has to decide which request tokens express the current intent, which memory identities matter, how far to traverse from them, and how much native K/V to activate. Searching too narrowly misses evidence. Searching and admitting everything spends compute and memory while increasing attention competition.

Consider the question “Which university did the founder of company X attend?” Company X identifies a root. That root exposes a founder or a new address. A successor search then finds the university. The root may be easiest to find lexically, while the successor relation may be semantic.

The request may need several roots but no graph traversal, or one root followed by one hop. These are different resource shapes, not merely different scalar difficulty levels.

PRA makes those shapes explicit. It separates logical identity discovery from physical native-K/V activation. This paper asks:

How should PRA allocate retrieval and native-K/V effort when different queries require different amounts and kinds of memory access?

A first implementation can expose three coherent profiles:

- E0 1 facet, 1 root, 0 hops, small budget,
- E1 2 facets, 2 roots, 1 hop, medium budget,
- E2 4 facets, 4 roots, 3 hops, large budget.

Profiles are easy to validate and safe to operate. They also couple controls unnecessarily. If a query needs one facet, four roots, and zero hops, a diagonal ladder must buy E2 even though most E2 resources are irrelevant. Factorization can choose that combination directly. This profile quantization problem motivates the paper’s main experiment before any learned controller is added.

The evidence supports two statements that must remain together:

factorized adaptive inference is useful,

but more controller complexity is not automatically useful.

The oracle surface contains much cheaper sufficient actions, but the present fine-grained routers mostly become cheap by failing to allocate enough. Independent root and successor representations also have large oracle headroom, yet the observable selector remains weak. Graph facets and one post-root callback broaden what the runtime can express without improving the deployable frontier.

This paper contributes:

1. a factorized PRA control model separating interpretation, logical search, and physical native-K/V admission;
2. a 792-action study that measures profile-quantization regret and finds roughly 40% lower oracle cost at matched quality;
3. a leakage-audited learned-controller evaluation showing that current fine-grained routers under-allocate and fail to capture that headroom;
4. independent root and successor retrieval-representation actions with measured method heterogeneity;
5. a strengthened graph-faceted request/reply baseline and a matched 29,700-action callback falsification study;
6. normalized conceptual and physical memory accounting, followed by a bounded native-K/V decode confirmation; and
7. a conservative runtime contract and explicit handoff to training, scaling, and serving work.

The study deliberately freezes model weights. Paper 4 asks whether PRA-aware training changes what is retrievable and predictable. Here the scientific object is adaptive inference over frozen routing and consumer behavior.

2 PRA as a Control Problem

2.1 Interpret, search, and admit

PRA control is easiest to understand as three decisions:

$$\boxed{\text{Interpret}} \rightarrow \boxed{\text{Search}} \rightarrow \boxed{\text{Admit}}.$$

Interpretation identifies the operative query region, constructs one or more facets, and chooses a root representation. Search retains roots, chooses a successor representation, and allocates breadth, depth, and a conceptual parent budget. Admission maps selected identities to native K/V, chooses granularity and consumer layers, and enforces a physical budget. A runtime may then evaluate observable uncertainty and make one bounded correction:

$$\text{Interpret} \rightarrow \text{Search} \rightarrow \text{Admit} \rightarrow \text{Evaluate} \rightarrow \text{Correct}.$$

In the running example, the company name can form an exact or BM25 root query; the selected company chunk exposes the founder; a semantic successor query follows that relation; and the admission step activates only the selected source spans in the model’s late attention layers. Logical search and physical disclosure remain separate even when they happen in one API call.

Only now do we write the action compactly:

$$a = (\mathcal{Q}, F_m, S_r, F, R, S_s, K, H, B_s, B_{kv}, L_c, G, M). \quad (1)$$

Here $\mathcal{Q}$ is the query region; F_m and F are facet mode and count; S_r and S_s are root and successor representations; R , K , and H are retained roots, successor breadth, and hop depth; B_s is the conceptual search budget; B_{kv} is the physical admission budget; and L_c , G , and M specify consumer layers, granularity, and disclosure policy. Thresholds and graph parameters are subordinate settings of the corresponding search or facet action.

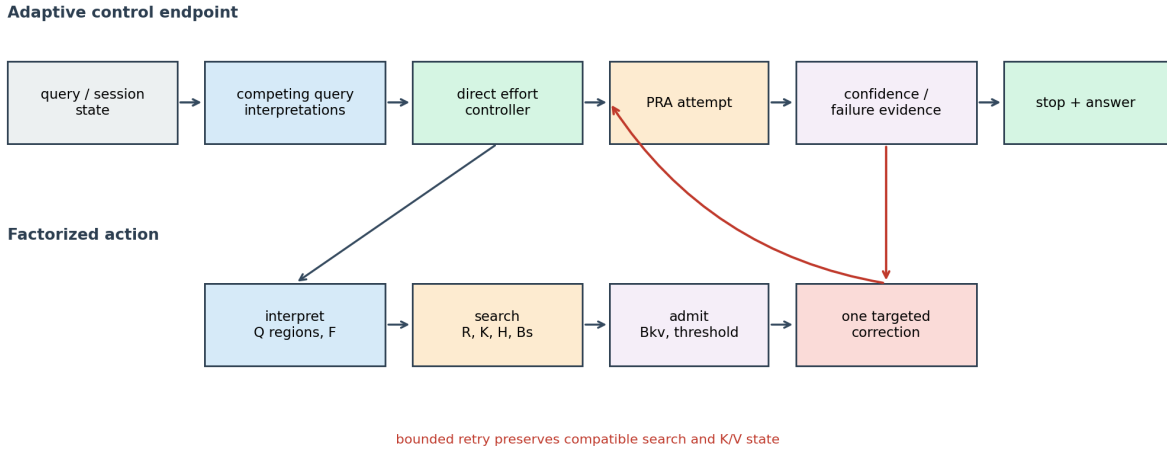


Figure 1: The adaptive endpoint. Interpretation, search, and admission are independent controls. Evaluation may authorize one bounded correction while preserving compatible identities and K/V. The callback path is an experimental hook, not the default policy.

2.2 Robust profiles and factorized actions

Table 1 gives the validation-frozen profile ladder. The profiles are not universal hyperparameters. They are a conservative SDK contract and a baseline against which independent actions can be measured.

Table 1: Validation-frozen effort profiles. Lower routing thresholds allow broader closure.

Profile	facets	R	K	H	B_s	B_{kv}	G	consumer layers
E0 low	1	1	2	0	2	256	256	27
E1 medium	2	2	4	1	4	512	128	26–27
E2 high	4	4	8	3	8	1024	64	24–27

Profiles increase every permission together. Factorized actions do not. They can spend on roots without buying more facets or hops, and can search broadly while admitting a compact physical set. This flexibility creates a larger prediction problem, so oracle headroom and learned policy quality must be reported separately.

2.3 Accounting without unit confusion

Let N be the available candidate parents, E the required evidence parents, and K the admitted parents. Evidence recall, precision, and complete recovery are

$$R_E = \frac{|K \cap E|}{|E|}, \quad P_E = \frac{|K \cap E|}{|K|}, \quad C_E = \mathbf{1}[E \subseteq K]. \quad (2)$$

K/N asks what fraction of available memory was activated. K/E asks how much memory was activated per required evidence item. Neither is a latency measure.

We keep five quantities distinct: search comparisons, unique conceptual candidates searched, conceptual candidates admitted, native K/V tokens or layer-token states, and wall-clock latency. Collapsing them into one number would make a posting-list lookup appear equivalent to a GPU dot product. The abstract cost used inside the factorized study is therefore an auditable ordering, not a serving-time claim.

3 Experimental Setup

The primary study reuses frozen Paper-2.5 native graph and query scores. Its grouped split contains 95 validation and 65 held-out HotpotQA/QASPER example–seed units. Validation supplies targets and thresholds; held-out oracle labels are evaluator-only. Complete evidence-chain recovery is primary.

The retrieval-representation study uses Paper-2.6’s 58 validation and 74 held-out identities across QASPER, HotpotQA, 2Wiki, and MuSiQue; 68 held-out examples contain a successor decision. The graph and callback experiment restores the same boundary and executes all 29,700 legal actions. Eight stratified examples check native-K/V materialization and frozen Qwen3-0.6B decode.

These scopes are not merged. The 65-unit endpoint is frozen-score chain recovery, the 74-identity endpoint is matched-budget evidence discovery, and the eight-example endpoint is a mechanism check. No generated-answer score is imputed for retrieval-only configurations.

Controllers may observe query features, score gaps, entropy, root competition, channel agreement, frontier shape, and selected counts. Dataset identity, gold evidence, oracle actions, answer

correctness, and evidence recall are rejected before serving feature construction. Five seeds measure initialization variability, not linguistic coverage.

The factorized study orders actions by

$$C(a) = F + N_{\text{transition}} + B_s + \frac{T_{kv}}{64}, \quad (3)$$

where T_{kv} is admitted source tokens. Artifacts also retain unscaled comparisons, searched and admitted parents, K/V tokens, and wall time. The cost is an auditable within-study ordering, not a serving-time surrogate.

4 Factorized Control: The Main Result

We replay

$$(F, R, K, H, B_s, B_{kv}) \in \{1, 2, 4\} \times \{1, 2, 4\} \times \{2, 4, 8\} \times \{0, 1, 2, 3\} \times \{2, 4, 8\} \times \{2, 4, 8\}. \quad (4)$$

Actions with $B_s < R$ or $B_{kv} < R$ are invalid, leaving 792 measured configurations per unit. Query-region policy and traversal order are fixed to isolate profile coupling.

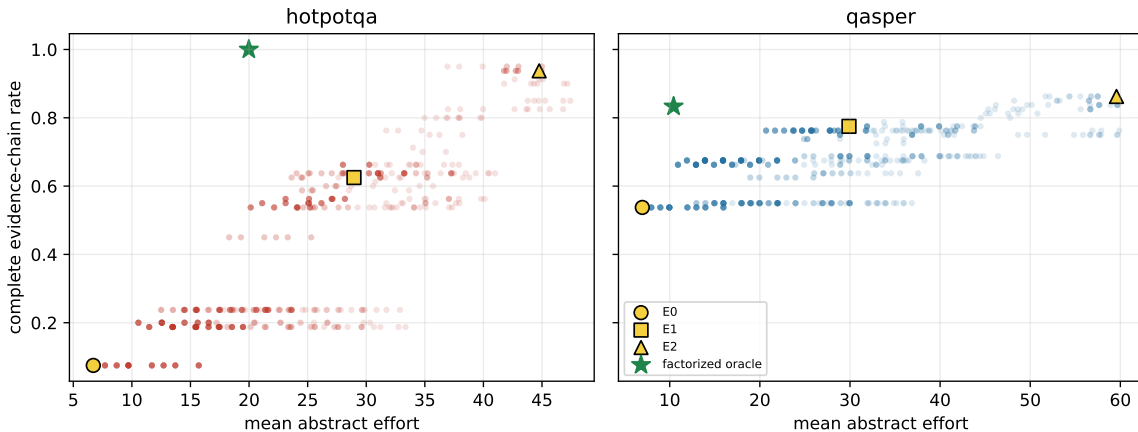


Figure 2: Held-out quality–cost surface. Stars are minimum-cost factorized oracles; E0–E2 are diagonal profile actions under the same accounting.

Table 2: Profile quantization regret. Matched rows contain units solved in both spaces.

Dataset	matched n	profile quality	factor quality	profile cost	factor cost	regret
HotpotQA	35	1.000	1.000	34.18	19.98	14.20
QASPER matched	24	1.000	1.000	16.11	8.95	7.15
QASPER all	30	.800	.833	16.82	10.43	—

At matched chain quality, factorization lowers mean HotpotQA cost by 41.5% and matched QASPER cost by 44.4%. It also solves one QASPER unit no profile solves. Difficult examples usually need one expensive control rather than E2 in full. One facet suffices for 97.1% of HotpotQA and 96.7% of QASPER units. HotpotQA’s additional work concentrates in root count, one graph hop, and parent budget; QASPER is root-only in 96.7% of units.

Table 3: Held-out candidate-normalized efficiency. Oracle and retry rows are evaluator ceilings; KV is mean admitted native-K/V tokens.

Dataset	Policy	R_E	P_E	C_E	K/N	K/E	KV
HotpotQA	R0 profile	.733	.400	.543	.697	1.905	979
	factorized oracle	.824	.624	.600	.576	1.633	840
	targeted retry	.948	.476	.886	.756	2.090	1079
QASPER	R0 profile	.750	.450	.700	.182	2.267	636
	factorized oracle	.917	.796	.833	.109	1.550	431
	targeted retry	.917	.588	.833	.190	2.517	675

QASPER supplies the cleanest sparse ceiling: the oracle raises recall and precision while admitting fewer candidates and 205 fewer K/V tokens than R0. HotpotQA is denser at this parent granularity; retry raises recall by activating more memory.

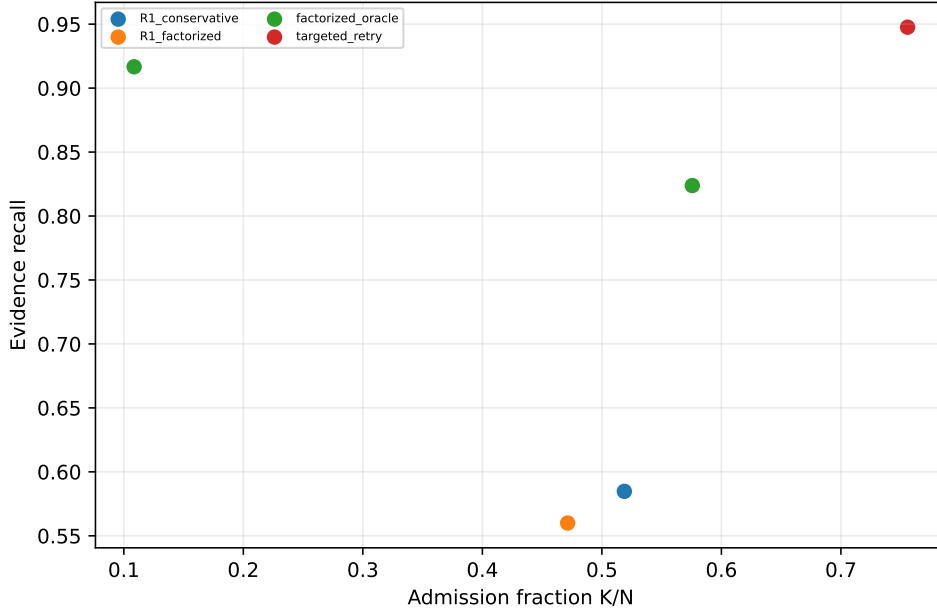


Figure 3: Evidence recall against admitted candidate fraction. Learned policies below R0 save cost by losing complete-chain quality.

5 Can We Predict the Cheap Sufficient Action?

Oracle optimization proves cheap actions exist; it does not make them observable. R0 predicts one profile. R1 predicts independent controls. Conservative R1 selects an upper quantile. R2 adds a signed-hash question representation. R3A predicts controls autoregressively.

Table 4: Representative held-out factorized-target controllers. Under-allocation means missing a chain completed by the factorized oracle.

Router	quality	cost	under	over
R0 profile	.631	24.29	.292	.523
R1 independent	.538	16.13	.385	.378
R1 conservative	.560	17.97	.363	.422
R2 conservative	.548	18.06	.375	.400
R3A autoregressive	.486	17.18	.437	.335
Factorized oracle	.923	15.57	.000	.000

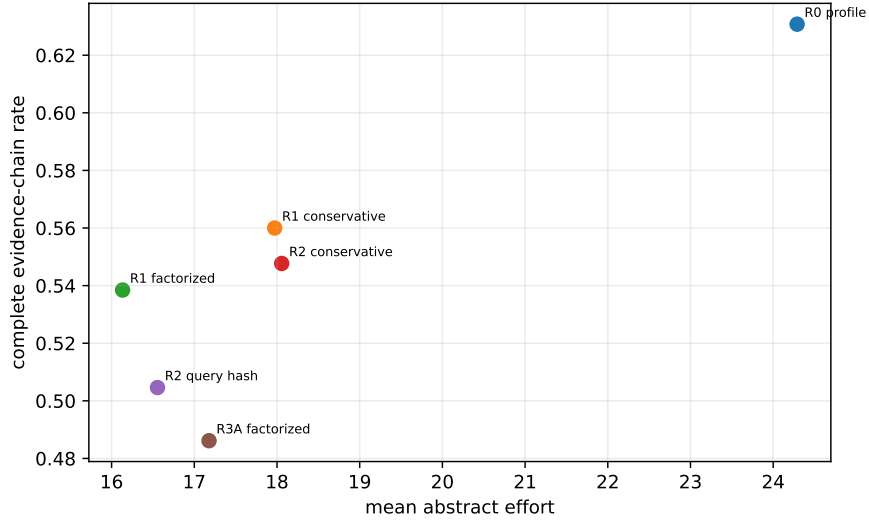


Figure 4: Learned quality versus expected cost. Apparently cheap fine-grained policies fail to allocate resources required by solvable examples.

No fine-grained router dominates R0. Conservative decoding recovers some quality over argmax, but lower cost remains coupled to under-allocation. R0 stays the supported initial controller despite its measurable quantization regret.

One evaluator-side retry exposes correction headroom. The cheapest successful single-control change or compound fallback raises aggregate quality from .631 to .923 at 2.49 mean added cost, correcting all 15 HotpotQA failures and four of nine QASPER failures without breaking a correct case. Eight repairs change only root or facet count. This is an oracle audit, not an online policy.

Model-native query representations do not rescue the controller. Grouped validation selects the last question-token embedding, whose held-out quality is .668 at cost 27.20, below observable profile routing at .702 and 28.25. Deeper hidden states, native Q/K, and a 256-token contextual diagnostic do not improve validation-selected control (Appendix C).

6 Retrieval Representation Is Also a Control Variable

Effort asks how broadly to search; representation asks what evidence relation defines the search. Root and successor methods independently choose semantic, exact, BM25, approximate, or hybrid variants. A semantic root may expose an exact address, so $S_r = S_s$ is not required.

The 74 held-out root oracles choose all five representations; none has majority support. Among 68 successor-bearing examples, 22 of 25 root-to-successor pairs occur. The most frequent cell contains only nine examples.

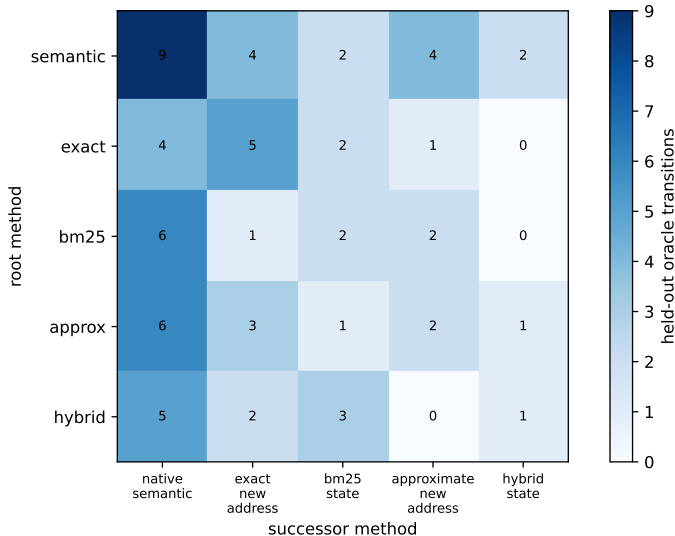


Figure 5: Per-example oracle transitions justify independent root and successor actions, but do not make the oracle observable.

Table 5: Matched two-root plus two-successor policies over 68 held-out examples.

Policy	interpretation	recall	precision	MRR
Fixed	BM25 root, native-semantic successor	.030	.081	.229
Static hybrid	one hybrid at both stages	.035	.088	.229
Learned joint	predicted root and successor	.032	.074	.225
Interpretable cascade	gap/disagreement rules	.048	.118	.240
Per-example oracle	best measured pair	.143	.353	.615

Representation heterogeneity is real; reliable selection is unresolved. The cascade is strongest overall but loses to the fixed pair on QASPER and MuSiQue. Leave-one-dataset-out method accuracy is .167–.278. Learned targeted retry raises .032 only to .037, while oracle correction reaches .143.

7 How Far Should Adaptation Go?

7.1 Query interpretation

In matched controls, explicit spans and structural query regions preserve root R@1 through 8,192 following payload tokens; a 16-token recent head fails after 32. Structural discovery is perfect on six semi-natural fixtures but falls to .400 under five role decoys, where recent-head remains correct. Explicit caller roles are preferred; automatic interpretations should compete under disagreement rather than unconditionally replace one another.

7.2 Semantic graph facets

Request/reply supports global, syntactic, multiscale, graph, and hierarchical syntactic-to-graph facets. Hierarchical nodes retain spans, centroids, entities, lexical and graph statistics, confidence, and parent/child provenance. Per-facet routing supports fixed, type-rule, learned, and oracle channel policies.

Graph facets are specialized, not universal. On the complete callback surface, the action oracle chooses global facets for 83.8% of examples, structural facets for 13.5%, and graph facets for 2.7%. Prototype graph construction costs 32–34 ms/example in exhaustive Python/CUDA replay. Detailed quality and construction rows are in Appendix E.

7.3 One root-state callback

The callback observes selected roots once and may choose a successor method or graph refinement. It cannot inspect dataset identity, evidence, or answers. A fair comparison crosses nine facet profiles with five root and five successor methods at the same two-root, two-successor, four-chunk budget:

$$132 \times 9 \times 5 \times 5 = 29,700 \text{ actions.}$$

Table 6: Matched test evidence recall. Learned and threshold rows report five-seed mean $\pm$ standard deviation. B6/B7 are evaluator ceilings.

ID	policy	recall
B0	validation-fixed global request/reply	.205
B1	query-only one-shot, no graph	.180 $\pm$.022
B2	query-only one-shot, graph available	.180 $\pm$.023
B3	root callback, no graph	.168 $\pm$.021
B4	root callback, graph initially available	.167 $\pm$.019
B5	learned conditional graph refinement	.167 $\pm$.021
B5t	threshold graph refinement	.188 $\pm$.024
B6	complete-action oracle	.365
B7	stage-wise oracle	.365

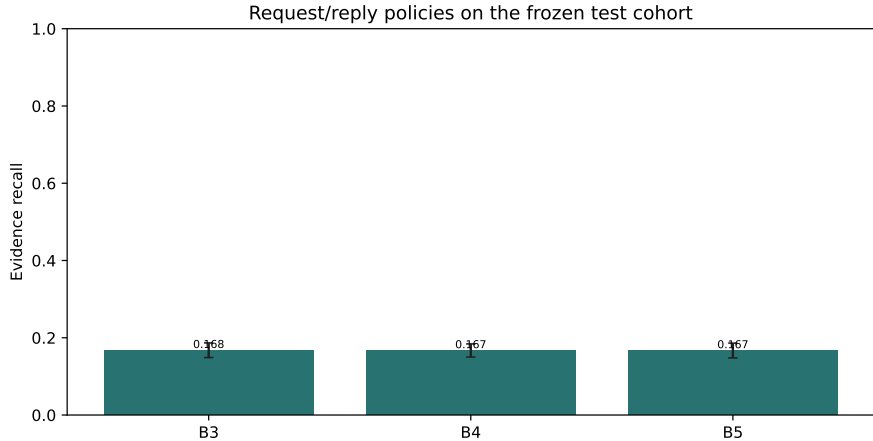


Figure 6: Added root-state complexity does not improve the supported retrieval frontier. Error bars reflect controller seeds, not example uncertainty.

Learned callbacks underperform matched one-shot control. Threshold refinement is somewhat better but remains below B0 and invokes graph on 16.5% of cases. Complete-action and stage-wise evaluator ceilings coincide at .3649 recall, .5777 precision, .7964 MRR, and .0270 complete recovery. Thus there is no additional stage-wise oracle advantage under these legal actions and supervision. This does not imply post-root state can never help under different representations, actions, training, or supervision.

8 Native-K/V Decode Confirmation

We map four conceptual chunks to 128 unique native-K/V tokens across Qwen layers 20–27 on two test examples per dataset. Every memory condition exposes 1,024 token-layer states (4 MiB), averaging 7.75% of source tokens.

Table 7: Eight-example frozen decode confirmation. $\Delta \log p$ is relative to no memory. This is a mechanism check, not benchmark QA.

Condition	retrieval recall	exact/F1	$\Delta \log p$	K/V tokens
No memory	.000	.375	.000	0
B0 fixed	.279	.375	+.567	128
B1 one-shot	.358	.500	+.770	128
B3 callback	.228	.375	+.912	128
B5t threshold	.360	.500	+.687	128
B6 oracle	.533	.500	+.639	128

Every memory row improves gold-token likelihood, confirming that materialized K/V affects the consumer. Exact/F1 is not monotonic in recall or likelihood. We therefore keep separate

retrieval quality $\neq$ likelihood utility $\neq$ generated answer quality.

9 Discussion

Lesson 1: factorization matters. Queries do not require one scalar effort level. Coherent profiles are robust operating points, but they should not define the full internal action space.

Lesson 2: prediction is harder than optimization. The oracle proves cheap sufficient actions exist. Current controllers do not identify them reliably; their lower cost is coupled to lower quality.

Lesson 3: more adaptation is not automatically better. Representation choices, graph facets, and callbacks enlarge the capability surface without producing a better learned deployable frontier. These negative results stop open-ended complexity.

Lesson 4: preserve optionality. Independent actions and one bounded callback hook remain useful architectural options because PRA-aware training may change the geometry. The supported frozen runtime remains conservative: explicit query roles, R0 initial effort, validation-fixed request/reply, independent budgets, observable traces, and at most one correction.

10 Related Work

External and iterative retrieval. RAG and DPR retrieve external text [12, 9]. MDR, IRRR, and IRCOT make retrieval state dependent [27, 18, 23]. Adaptive-RAG selects no, single, or iterative retrieval [7]; FLARE retrieves during generation [8]; Self-RAG trains retrieval and reflection decisions [1]. PRA instead controls query representation, persistent identity traversal, and native-K/V admission.

K/V selection and sparse attention. H2O, SnapKV, StreamingLLM, Quest, KIVI, and ClusterKV select, index, evict, or compress physical K/V [29, 13, 25, 22, 15, 14]. PagedAttention manages storage [11]. PRA separates upstream logical discovery from physical activation; paging, compression, and eviction remain complementary.

Longformer, BigBird, Reformer, and Routing Transformer change architectural sparsity [3, 28, 10, 21]. RETRO, Memorizing Transformers, Compressive Transformer, Infini-Attention, Landmark Attention, and Titans provide external, recurrent, compressed, or learned memory [4, 24, 19, 17, 16, 2]. We do not imply equivalence to a frozen PRA control layer.

Adaptive computation. Adaptive Computation Time learns halting [6]; early exit allocates depth [26]; sparse experts allocate parameters [5]; and Mixture-of-Depths allocates layer compute across tokens [20]. PRA controls a different object: interpretation, retrieval representation, traversal, and K/V disclosure, whose costs cannot be reduced to one uniform operation count.

11 Limitations

The primary endpoint is frozen-score chain recovery, not generated answer quality; the live decode check has eight examples. The factorized cohort has 65 held-out example-seed units but only 32 unique questions. The channel study has 74 held-out identities and 68 successor labels. Five seeds do not replace larger identity cohorts or cross-model replication.

Equation 3 orders actions within this study. Python/CUDA graph timings and inherited CPU systems measurements are not production latency. Automatic query interpretation is heuristic. Learned controllers use small validation sets; strong retry and action oracles use evaluator outcomes and cannot be deployed.

The callback surface fixes two roots, two successors, and four admitted chunks. It does not cross hop depth, variable admission, or jointly trained root-state representations. Equal complete and stage-wise ceilings apply only to this Cartesian surface. No PRA-aware backbone is trained here.

12 Conclusion

What did we learn? Factorized controls expose cheaper sufficient PRA actions: HotpotQA cost falls from 34.18 to 19.98 and matched QASPER from 16.11 to 8.95 at matched chain quality.

What did not work? Fine-grained routers, model-native self-routing, learned representation selection, graph control, and callbacks do not beat the robust conservative policy. Their oracle headroom is real; their observable prediction is not reliable enough.

What follows? Paper 4 studies PRA-aware training, Paper 5 scaling, Paper 5.5 runtime productization, and Paper 6 first-class vLLM integration. This paper closes frozen adaptive inference with a factorized action space, a conservative default, and a clear stopping point for controller complexity.

A Query-Region Diagnostics

The matched corpus crosses 24 cases, three payload forms, and five serializations while holding evidence, eight candidate roots, and 16 active K/V tokens fixed.

Table 8: Root R@1 / answer proxy under matched layouts.

Layout	recent head	explicit	structural	head+reinterpret
$C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
$Q + C$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00
$C_1 + Q + C_2$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00
$I + C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
Q +long payload	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00

Head R@1 averages .400; explicit and structural average 1.000. Reinterpretation corrects all 216 head failures and breaks none of 144 correct cases. Broad all-region selection reaches .333.

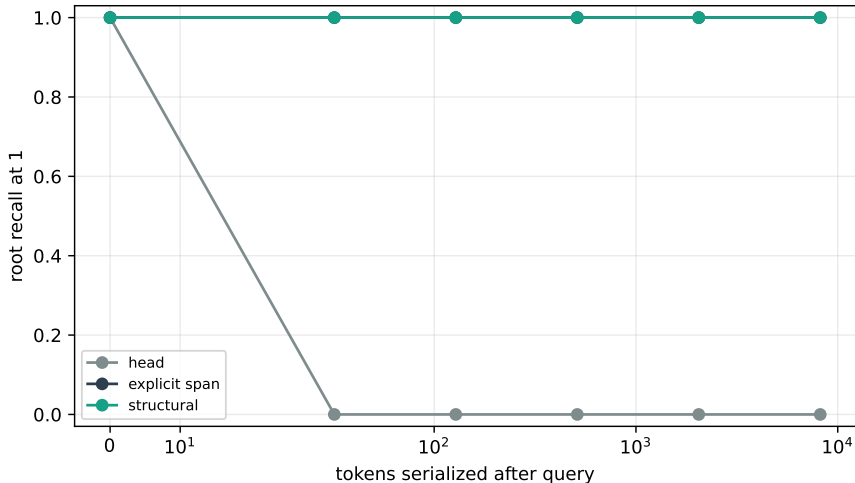


Figure 7: Recent-head failure under displacement; explicit and structural spans remain stable.

Structural discovery reaches R@1 1.000 and span IoU .981 on six semi-natural fixtures. Under five role decoys, it falls to .400 and .314. Session, region-count, facet, URL-role, and retry details remain in the artifacts.

B Preliminary Profile-Derived Routers

Table 9: Held-out routers trained on inherited profile labels.

Router	quality	cost	under	over
R0 profile	.615	109.9	.031	.169
R1 feature MLP	.594	103.7	.055	.166
R2 signed-hash MLP	.591	104.0	.046	.157
R3A autoregressive	.591	102.6	.049	.145

These labels inherit profile coupling and are superseded by the factorized-target experiment. Added capacity saves nominal cost while losing quality.

C Self-Encoded Query Routing

Qwen3-0.6B is frozen at revision `c1899de289a04d12100db370d81485cdf75e47ca`. The sweep pools the explicit question span from embeddings, hidden layers, and pre-RoPE native Q/K. Projection and selection use grouped validation.

Table 10: Representative five-seed profile routing by representation.

Representation	CV quality	test quality	total cost	under
Observables	.728	.702	28.25	.222
Signed hash	.686	.609	25.53	.314
Embedding last (selected)	.726	.668	27.20	.255
Embedding mean	.699	.717	25.74	.206
Hidden layer 7 mean	.714	.698	27.13	.225
Native K layer 13 mean	.718	.655	27.85	.268
Contextual layer 14 mean	.705	.674	27.70	.249

Embedding mean is numerically highest on test but was not validation-selected. Contextual states cost more and do not improve selected control. Exact prefix continuation before the first PRA consumer reuses the state without changing quality.

D Inherited Runtime and Proxy Measurements

These are provenance for Paper 5.5, not Paper 3.5 contributions. At 4,096 keys, exact GEMM reduces a Python Top-8 loop from .827 s to .327 ms with parity. The coarse index reaches .332–.648 recall and takes 8–13 ms, so it is not adopted. CPU gather shows no stable speed advantage; Python page lookup is 14–37 times slower. At concurrency 32, padding wastes 44.1% of slots. None is a production GPU throughput result.

The controlled 1,024-document, 200-query corpus forces a two-hop bridge. One-shot RAG and fixed low PRA recover .005; iterative RAG and adaptive medium PRA recover 1.000 with controlled active-token counts 320 and 192. The construction demonstrates accounting and iteration, not general superiority. Mechanism-faithful K/V proxy rows are likewise retained as bookkeeping controls, not system rankings.

E Graph Diagnostics

Table 11: Global graph versus hierarchical syntactic-to-graph root discovery.

Dataset	graph R	hier. R	graph work	hier. work	graph ms	hier. ms
QASPER	.033	.064	90	90	31.9	18.8
HotpotQA	.234	.212	524	524	70.2	35.5
2Wiki	.175	.175	288	288	40.1	31.1
MuSiQue	.090	.092	796	570	39.1	36.1

Adding graph to inherited hybrid changes recall by $-.0078$, $+.0156$, zero, and $-.0078$ on QASPER, HotpotQA, 2Wiki, and MuSiQue. Per-facet fixed, type-rule, learned, and oracle policies reach $.120$, $.224$, $.198 \pm .020$, and $.283$ recall on an explicitly post-hoc exploratory split. Graph calls, density, components, overlap, and facet assignments remain public.

F Callback Ablations

Adding the root gist raises learned B5 recall from $.167$ to $.177$, still below B1. Threshold control corrects 21.6% and harms 17.6%; three of five seeds invoke no graph. Leave-one-dataset-out B1, B2, B3, and B5t reach $.167$, $.170$, $.155$, and $.180$ versus $.190$ fixed and $.365$ oracle. Full seed behavior, feature ablations, correction/harm rows, and transfer tables remain in the result package.

G SDK and Artifact Manifest

An independent implementation should preserve five invariants: separate conceptual and physical budgets; reject evaluator labels before vectorization; reuse compatible identity and K/V state while accounting for regeneration; label component versus end-to-end timing; and keep query-role discovery separate from faceting with explicit caller spans authoritative.

Core contracts are in `src/prahf/adaptive_runtime.py`, `src/prahf/factorized_control.py`, `src/prahf/adaptive_search.py`, `src/prahf/adaptive_facets.py`, `src/prahf/facet_routing.py`, and `src/prahf/root_callback.py`. Drivers under `experiments/paper3_5_adaptive_pra` generate the profile, factorized, normalized-efficiency, self-router, graph, callback, and decode artifacts.

All normalized outputs are under `docs/papers/shared/results/paper3_5_adaptive_pra`. They include identity partitions, action specifications, controller seeds, calibration, oracle and Pareto rows, allocation outcomes, candidate/KV accounting, graph construction and overlap, callback ablations, decode rows, findings JSON, plots, and claim audits. The 29,700-action surface and 5,940 persisted root states support policy replay without repeating retrieval. The README lists complete study commands.

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.
- [2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [6] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [7] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 7036–7050. Association for Computational Linguistics, 2024.
- [8] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992. Association for Computational Linguistics, 2023.
- [9] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
- [10] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [13] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [14] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.
- [15] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [16] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.
- [17] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- [18] Peng Qi, Haejun Lee, Timo Sido, and Christopher D. Manning. Answering open-domain questions of varying reasoning steps from text. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3599–3614. Association for Computational Linguistics, 2021.

- [19] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [20] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [21] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
- [22] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [23] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.
- [24] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [26] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2246–2251. Association for Computational Linguistics, 2020.
- [27] Wenhan Xiong, Xiang Lorraine Li, Srinivasan Iyer, Jingfei Du, Patrick Lewis, William Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, and Barlas Oguz. Answering complex open-domain questions with multi-hop dense retrieval. In *International Conference on Learning Representations*, 2021.
- [28] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [29] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.